Problème

- Planifier la maintenance de 7 centrales électriques sur 4 intervalles.
- **But :** maximiser la puissance en réserve pendant l'année.
- **Capacités (MW) :** 20, 15, 35, 40, 15, 15, 10 (total = 150).
- **Unités 1 et 2 :** 2 intervalles de maintenance.
  Unités 3 à 7 : 1 intervalle.
- Demande par intervalle : 80, 90, 65, 70 MW.
- **Contraintes :** la maintenance commence au début d'un intervalle et finit à la fin du même intervalle, et la réserve doit être supérieure à la demande.
- **Réserve = capacité totale - puissance en maintenance - demande.**

Encodage

Chaque unité a un **gène de 4 bits** (1 = en maintenance, 0 = non), un **bit par intervalle**.

Unités 1 et 2 : 1100, 0110, 0011.
Unités 3 à 7 : 1000, 0100, 0010, 0001.

Un chromosome = 7 gènes de 4 bits.

- Il faut maintenant trouver un encodage pour problème et ses contraintes.
- Dans ce cas-ci, on utilise des gènes à 4 bits intervalles de maintenance.

Unité	Maintenance possible
1	1100, 0110, 0011
2	1100, 0110, 0011
3	1000, 0100, 0010, 0001
4	1000, 0100, 0010, 0001
5	1000, 0100, 0010, 0001
6	1000, 0100, 0010, 0001
7	1000, 0100, 0010, 0001

Données du problème

Unité	Capacité (MW)	Nombre d'intervalles nécessaire pour maintenance
1	20	2
2	15	2
3	35	1
4	40	1
5	15	1
6	15	1
7	10	1

Centrales, leur capacité, et le nombre d'intervalles de maintenance requis.

- On sépare l'année en 4 intervalles; la demande en puissance pendant ces intervalles est: 80, 90, 65 et 70 MW, respectivement.
- D'autres contraintes:
  - La maintenance d'une unité commence au début de l'intervalle et se termine à la fin de l'intervalle.
  - La réserve de puissance pendant un intervalle doit être plus grande que la demande en puissance.
- On veut optimiser la réserve de puissance pendant les intervalles.
  - Réserve = Capacité totale – (P_{perdue} maintenance) - demande

```
CAP = [20, 15, 35, 40, 15, 15, 10]   # capacité (MW) de chaque unité
DEMANDE = [80, 90, 65, 70]           # demande par intervalle
TOTAL = sum(CAP)                     # 150 MW

def reserves(chrom):
    res = []
    for i in range(4):
        perdue = 0
        for u in range(7):
            if chrom[u][i] == "1":
                perdue = perdue + CAP[u]
        reserve = TOTAL - perdue - DEMANDE[i]
        res.append(reserve)
    return res
```

- `chrom[u]` est le gène de l'unité u, par exemple "0110".
- `chrom[u][i]` est le bit numéro i dans le gène. S'il vaut "1", l'unité est en maintenance pendant l'intervalle i.
- `CAP[u]` est la capacité de l'unité u en MW.

Fonction objectif

```
def fitness(chrom):
    liste_reserves = reserves(chrom)
    plus_petite = min(liste_reserves)
    if plus_petite < 0:
        return 0
    return plus_petite
```

- réserve de chacun des 4 intervalles
- la plus petite réserve
- si elle est négative `return 0` (solution invalide, fitness = 0)
- sinon `return` la fitness

Exemple : reserves = [20, 25, 35, 30]
min = 20
20 n'est pas inférieur à 0, donc **return 20**

Exemple d'une solution invalide : reserves = [-5, 25, 35, 30]
min = -5
-5 est inférieur à 0, donc **return 0**

```
def selection_loterie(pop, fits):
    total = sum(fits)
    if total == 0:
        return random.choice(pop)
    r = random.uniform(0, total)
    cumul = 0
    for ind, f in zip(pop, fits):
        cumul += f
        if cumul >= r:
            return ind
    return pop[-1]
```

Croisement

Croisement simple

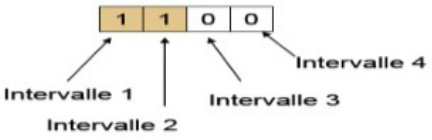
On choisit au hasard un point de croisement, pour chaque couple.

```
def croisement(p1, p2):
    if random.random() < PC:
        pt = random.randint(1, 6)
        return p1[:pt] + p2[pt:], p2[:pt] + p1[pt:]
    return p1[:], p2[:]
```

Mutation

Inversion d'allèles : cette mutation consiste à choisir deux allèles au hasard et d'inverser l'ordre des allèles contenus dans la zone sélectionnée.

- Encodage, ex: pour l'unité 1, le 1100 veut dire qu'il est en maintenance pendant les intervalles 1 et 2.



« 1 » veut dire maintenance,
« 0 » veut dire pas de maintenance

- Il faut maintenant définir la fonction objective.
  - On fera la somme de la puissance des unités qui sont en maintenance, et on calculera la réserve nette.
- Exemple: Soit un chromosome quelconque:

0110	0011	0001	1000	0100	0010	1000

- La somme de la puissance des unités en maintenance est:

$$\text{Intervalle 1} : 0\times20+0\times15+0\times35+1\times40+0\times15+0\times15+1\times10=50$$
$$\text{Intervalle 2} : 1\times20+0\times15+0\times35+0\times40+1\times15+0\times15+0\times10=35$$
$$\text{Intervalle 3} : 1\times20+1\times15+0\times35+0\times40+0\times15+1\times15+0\times10=50$$
$$\text{Intervalle 4} : 0\times20+1\times15+1\times35+0\times40+0\times15+0\times15+0\times10=50$$

```python
def mutation_inversion(chrom):
    c = chrom[:]
    if random.random() < PM:
        groupe = random.choice(GROUPES)
        debut = groupe[0]
        fin = groupe[1]
        i = random.randint(debut, fin)
        j = random.randint(debut, fin)
        if i > j:                # on s'assure que i <= j
            i, j = j, i
        zone = c[i:j + 1]        # la zone entre les deux positions
        zone.reverse()           # on inverse l'ordre des gènes
        c[i:j + 1] = zone        # on remet la zone dans le chromosome
    return c
```